



COMPSCI 4ZP6A slide set 3

Proof of concept,
Design + V&V document,
Final submission,
Hazard management,
Software license

Capstone project
2026

Resource support from the department

- The department of Computing and Software maintains its own server and end user computing infrastructure. Students have access to virtual desktop cluster from terminals in labs as well as their own devices. On the server side, GPU cluster is available for LLM computing, as well private cloud for groups that require a virtual server for their project. Moreover, CAS has set up organizational accounts with both the Apple App Store and Google Play for use within your Capstone course. For more details, please visit the CAS [tech support webpage](#). [Only works on campus or on VPN]
- CAS will reimburse groups for approved expenses, up to a maximum of \$150/group. All expenses submitted for reimbursement must be approved by your TA. Reimbursements requests will be sent to casadmin@mcmaster.ca and must include the proof of the TA's written approval, all necessary receipts (including HST number) and a CC to [name of instructor]. CAS will cover the cost of printing EXPO posters up to \$60/group, details to come later.
- Please fill the form I will post on teams and Avenue.

Reminder

- **Members of the same team: Please submit identical PDF files with the same file name for project / team form.**
- **For future submission, you will submit per team**

We covered

- SRS + Development plan
in previous slides.

- SRS is a list of requirements with priority levels and project plan is your plan to execute. The two will be submitted as one document.

We continue going over the other required documentations in this lecture.



Proof of concept presentations

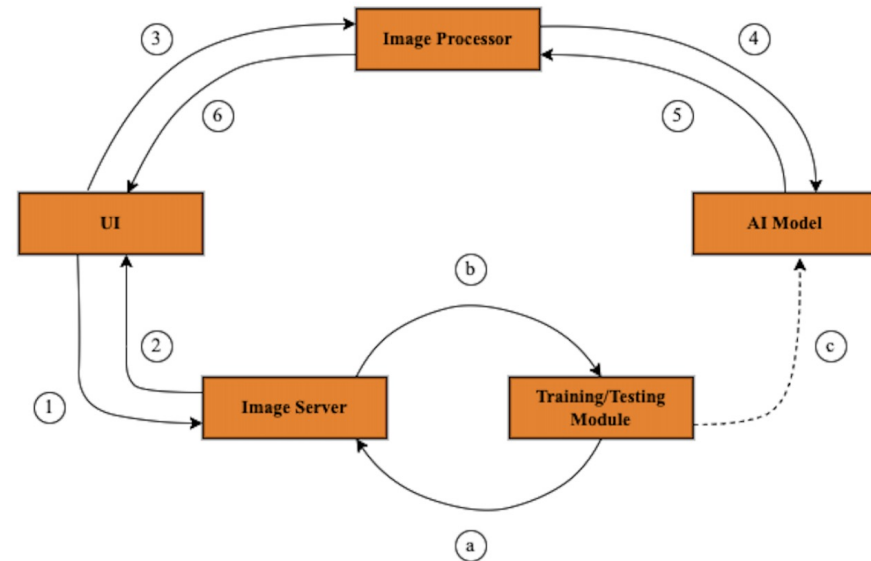
- Logistics:
 - Nov 23 (mandatory class, we take attendance, **we use both tutorial and class time.**)
 - Each group prepares a 2-minute video due on Nov 22, upload a video file on Avenue, I will download all and play on my computer in class.
 - We call groups to stage and play the videos in class followed by 1 question and move on. Your instructor and TAs might ask for clarification afterwards.
- Have a plan from the SRS stage as to what you will show. It will demonstrate implementation.
- Components to cover in your video:
 - a. A single slide with voice over that covers the followings in about 1 min:
 1. Brief problem statement (purpose)
 2. Key features of the solution and stakeholders
 - b. A demonstration that should clearly exhibit meaningful progress in about 1 minute (record off your laptop screen), need to see evidence of working code. It cannot be slides. Show that P0 features from the SRS are being implemented. If P0 features are not clearly identified in SRS, they should be clarified here
- Example: for an image classification application, show that you have a working model that you can call and get a label back plus a mock up of your future UI.

Design + V&V document

- Your SRS identifies “What”, design identifies “How”
- Design should meet functional and non-functional requirements
- Separation of concerns is required: **modularization**
- Documentation improves maintainability, reusability, understandability
- Document so can write test cases later in V&V

System Architecture

- Provide an overview of how the functionality and responsibilities of the system were divided and assigned to subsystems.
 - subsystems 1
 - subsystems 2
- Example: For an AI app that works on data to produce some insight:
 - Data portal
 - ML backend
 - Frontend
- Often a **component diagram** is appropriate



- 1) An image is requested from the image server.
- 2) The requested image is sent to the UI.
- 3) The image is sent to the image processor to be prepared for the model. Image preparation includes converting the image from DICOM format to JPEG format, fixing the color channels, resizing the image, and converting the image to a tensor.
- 4) The processed image is sent to the model.
- 5) The results from the model are sent back to the image processor to be prepared for the UI.
- 6) The results are reformatted to be displayed on the UI.

Detailed Modular System Design

- Describe components as software modules (might be more granular than subsystems, for example, the AI backend itself can be broken down to multiple modules).
- A software module has two components
 - An **interface** that enables the module's clients to use the service the module provides
 - An **implementation** of the interface that provides the services offered by the module

The Module Interface (API)

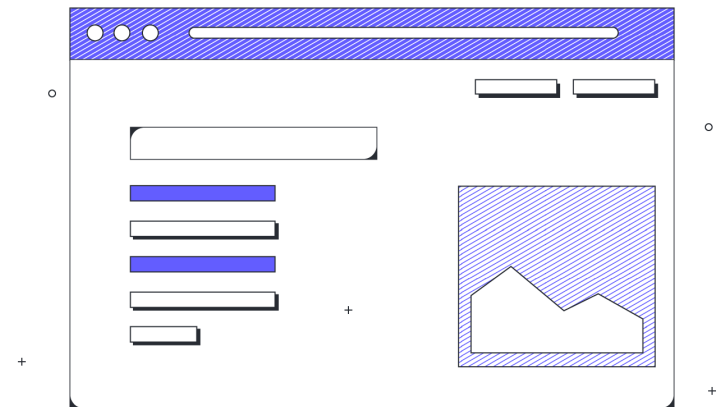
- A module's interface can be viewed in various ways
 - As a set of services
 - As a contract between the module and its clients
 - As a language for using the module's services
 - An interface describes the data and procedures that provide access to the services of the module

The implementation

- The implementation is hidden from other modules
- The interface data and procedures are implemented together and may share data structures
- The implementation may utilize the services offered by other modules
- Encapsulate changeable design decisions as implementation secrets within module implementations
- ***Include class diagram, include class members and variables***
 - **Example:** The ML model should have an API that is consistent with various ML implementations. Changing the 'secret' [implementation] of the model should not change the way other modules interact with it.

Graphical user interface (GUI)

- Design of front end
 - What the user sees and interacts with
 - Description, sketches, **wireframe**.
 - Design philosophy (simplicity?)
 - Expected look and feel



A large orange rounded rectangle on the left side of the slide.

V&V plan

- New in 2025: This will be part of the design document, a short 2-3 page section.



V&V plan

What does it involve?

- Set of unit tests of software components
 - Add “report”=test results at the time of final submission
- Validation of GUI/frontend could be based on user experience questionnaire
- If you are building a system with performance targets, include testing strategy
 - For AI/ML systems, describe your **testing strategy, hold out set and performance metrics** you will report and why they are appropriate. [I will define these and teach best practices in the next lecture]
 - Add the metric values in the final report



Minimum requirements for the submission of the design + V&V document for this course

- Brief (one paragraph) purpose statement for the project and for the document.
- Component diagram: Components and their relationships. If any component is complex, consider a secondary chart for more details for that component. Example for an AI based model with a front end:
 - a. Training module
 - b. Image server
 - c. Deployed AI model and logic (backend)
 - d. UI (front end)
- Relationship between components and requirements: Describe what requirement each component meets. This might not be a 1 to 1 relationship.
- For each component, describe:
 - **Normal behavior:** What is the expected behavior from each component.
 - API (input/outputs with emphasis on data structures being exchanged between components).
 - Implementation (Include details: classes, methods, and their relationships).
 - Potential undesired behaviours (this would be the place to think what can go wrong. Think simultaneously about testing and verification here. Although the details on this topic will go into V&V).
- Details on user interface: Describe the UI design, user interactions, placement of items, and color/font choices. Schema charts or Figma are encouraged.

Minimum requirements for V&V in the Design + V&V doc

Component Test plan: For every component of the system, provide one or both of the followings, based on the advice below:

- 1) Unit tests:** A testing methodology that verifies desired behavior and lack of undesired behaviors which you identified in your design. Typically, this will be unit tests. Any software component needs testing (both frontend and backend).
- 2) Performance tests and metrics:** A metric and measurement methodology to verify your claims in terms of performance. Note that performance is not only for ML systems. If you have a web-based system, measuring the latency is important.

Hazard Analysis

- Formal hazard analysis document is not a required component of this course.
 - See the very interesting talk from Professor Alan Wassying on the topic of diversity and redundancy in software systems (on Teams general channel).
- But protection against hazards is required and has to be built into SRS and design docs.
- What is hazard?
 - It's a property or condition in the system together with a condition in the environment that has the potential to cause {harm or damage} = loss



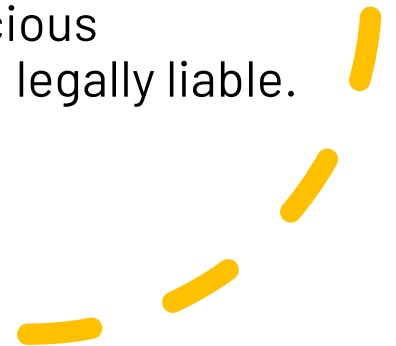
Hazard Analysis

- All projects with a web component, or user facing applications, are expected to adhere to cyber-security standards.
- Protection of user data and personal information is a must
- Secure payment systems if you process payments
- Example in healthcare:
 - Protected health information (PHI) is subject to HIPPA law in USA and PIPEDA in Canada
 - Access needs to be 'need-based' and logged.
- Any system with users
 - User authentication, solutions for forgotten password



What is a software license?

- A software license is a legal agreement that defines the terms of usage and provides the right to one or more copies of software to end users. Without a license, using and redistributing the software would be considered a breach of copyright law.
- If your Git repository is open, please post a license. That means specify what licence governs the use of your code by others. This is not just to protect your intellectual property. But also to protect you against abuse of your software for malicious purposes which could make you legally liable.



Types of license

	Public domain & equivalents	Permissive license	Copyleft (protective license)	Noncommercial license	Proprietary license	Trade secret
Description	Grants all rights	Grants use rights, forbids almost nothing (allows proprietization, license compatibility)	Grants use rights, forbids proprietization	Grants rights for noncommercial use only. May not be combined with copyleft.	Traditional use of copyright ; no rights need be granted	No information made public
Software	PD, CC0	BSD , MIT , Apache	GPL , AGPL	JRL , AFPL	proprietary software , no public license	private, internal software

MIT license is appropriate for most of you

- Copyright (c) <year> <copyright holders>

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Final submission

- The final deliverables for this course consist of three components:
- **1) Demonstration in class with a recorded 2 minute video:** Detailed instructions on this component will be provided. File upload on A2L: **April 5**, present in class on **April 4, 2026**
- **2) Final submission of code and all documents and a PDF poster on GitHub. April 5, 2026.** Include:
 - Final version of SRS, Project Plan, Design Doc and V&V document which is amended with test results report
 - One page '**completion report**'. List requirements from SRS, their priority, and completion rate (0-100%). If P0 or P1 requirements were not completed explain why.
 - Code
 - Readme file with clear instructions for code so a user can run
 - Should include a link to website/app, or clear application download instructions for executable file
 - TAs should be able to try the application without compiling your code
 - They need to be able to log in and try
 - Poster (PDF). Size and template for this will be communicated shortly by the organizers of the Faculty of Engineering Capstone Project Expo
- **3) Poster presentation at the Faculty of Engineering Capstone Expo:** Time and logistics to be communicated by the organizers.